

NOM	CACHADA CAPITAO
Prénom	Estefania
Date de naissance	02/02/1987

TP – Développeur Web et Web Mobile

Lien du git : <https://github.com/Fani1987/EcoRide>

Lien de l'outil de gestion de projet : <https://trello.com/b/rPsh9uZc>

Lien du déploiement : ecoride-production-d9b8.up.railway.app

- Login administrateur : admin@ecoride.fr
- Mot de passe admin : admin1234

Partie 1 : Analyse des besoins

1. Présentation du projet :

EcoRide est une plateforme web de covoiturage écologique visant à connecter des conducteurs et des passagers dans une logique de mobilité durable. L'objectif est de favoriser les trajets en véhicules respectueux de l'environnement, tout en proposant une interface intuitive et sécurisée. L'application gère des utilisateurs aux rôles multiples (passager, chauffeur, employé, admin) et repose sur un système économique de crédits.

Le cahier des charges inclut :

- Un système d'authentification et des espaces dédiés pour chaque rôle.
- La gestion complète du cycle de vie d'un trajet (proposition, réservation avec confirmation, démarrage, fin, annulation).
- Un système de notation et de modération des avis.
- Une architecture de données hybride utilisant à la fois MySQL et MongoDB.
- Un tableau de bord administrateur avec visualisation de statistiques

2. Compétences du référentiel couvertes

Ce projet couvre les compétences des deux activités types du Titre Professionnel :

Activité type 1 : Développer la partie front-end

- **Maquetter** : Des maquettes ont été réalisées pour les pages principales (Accueil, Profil, Admin).
- **Réaliser des interfaces statiques** : Les maquettes ont été intégrées en HTML5 sémantique et Sass (SCSS), en utilisant le framework Bootstrap 5.
- **Développer la partie dynamique** : Le JavaScript natif a été utilisé pour les appels fetch asynchrones (réservations, graphiques admin) et la manipulation du DOM (gestion des modales).

Activité type 2 : Développer la partie back-end * Mettre en place une BDD relationnelle :

Une base MySQL a été conçue pour gérer les utilisateurs, les trajets et les réservations .

- **Développer des composants d'accès SQL et NoSQL** : L'accès aux données est sécurisé via PDO (requêtes préparées) pour MySQL et la bibliothèque MongoDB PHP pour les préférences utilisateur (NoSQL) .
- **Développer des composants métier** : La logique de gestion des sessions, des rôles, des transactions (réservation, paiement) et le routage ont été implémentés en PHP natif via une architecture Modèle-Vue-Contrôleur (MVC) .

3. Objectifs du projet

- ✓ **Mobilité durable** : Promouvoir les trajets écologiques.
- ✓ **Fiabilité** : Assurer un cycle de vie transactionnel sécurisé pour les réservations et les crédits.
- ✓ **Transparence** : Mettre en place un système d'avis et de notation modéré.
- ✓ **Pilotage** : Fournir à l'administration des outils de visualisation des statistiques.

4. Description des utilisateurs (Personas)

- ✚ **Le Visiteur** : Tout public. Veut pouvoir rechercher un trajet rapidement sans s'inscrire.

- ✚ **Le Passager** : Utilisateur connecté. Veut réserver un trajet, gérer ses réservations, et être rassuré par les avis sur les chauffeurs.
- ✚ **Le Chauffeur** : Utilisateur connecté. Veut proposer un trajet, gérer les demandes de ses passagers et être payé (en crédits) à la fin du trajet.
- ✚ **L'Employé** : Personnel interne. A besoin d'une interface simple pour modérer les avis et gérer les litiges (incidents) signalés.
- ✚ **L'Administrateur** : Super-utilisateur. Doit pouvoir gérer les comptes (suspension, création d'employés) et surveiller l'activité du site.

Partie 2 : Spécifications technique

Cette section résume les choix d'architecture, les technologies et les méthodes de travail employées pour le projet EcoRide.

1. Stack Technique

- **Backend** : PHP 8.1+ (Architecture MVC/Contrôleur Frontal)
- **Frontend** : HTML5, SCSS (Sass), JavaScript (ES6+), Bootstrap 5
- **Bases de Données** : Hybride (MySQL/PDO + MongoDB)
- **Gestion des Dépendances** : Composer (PHP) et NPM (JS)
- **Déploiement** : Railway.app (PaaS)
- **Librairies Clés** : PHPMailer, Dotenv, Chart.js

2. Justification des Choix d'Architecture

Chaque technologie a été choisie pour répondre à un besoin précis du projet.

⚙️ Backend (Architecture MVC)

L'application repose sur un **Contrôleur Frontal** (`index.php`) qui dirige toutes les requêtes vers les bons contrôleurs. Cette architecture, inspirée du **modèle MVC**, a été choisie pour :

- **Démontrer la maîtrise des concepts fondamentaux** de PHP (routage, PDO, gestion des sessions) sans framework.

- Assurer une **séparation claire des responsabilités** : les Modèles gèrent le SQL , les Vues gèrent l'affichage, et les Contrôleurs gèrent la logique métier (transactions, sécurité).

Bases de Données (Persistance Polyglotte)

Une approche hybride a été utilisée pour les données:

- **MySQL (via PDO)** a été choisi pour sa robustesse et ses garanties **ACID** (transactionnelles). Il gère les données critiques et relationnelles : utilisateurs, trajets, et réservations.
- **MongoDB** a été utilisé pour sa **flexibilité**. Il gère les préférences utilisateur, qui sont des données non structurées. Sa nature "sans schéma" permet aux utilisateurs d'ajouter des préférences personnalisées sans modifier la structure de la base .

Frontend (Interface Dynamique)

- **Bootstrap 5** a été utilisé pour développer rapidement une interface responsive (grille, composants de modale) .
- **Sass (SCSS)** a été utilisé pour rendre le CSS maintenable via des variables, des "partials" et l'imbrication .
- **JavaScript Natif (ES6+)** a été privilégié pour sa légèreté. Il gère les interactions dynamiques (modales) et les appels API asynchrones (fetch) vers le backend, notamment pour les réservations et les graphiques du dashboard admin.

3. Sécurité (Basée sur l'OWASP)

La sécurité a été un pilier central, en réponse aux failles principales de l'OWASP :

- **Prévention des Injections SQL** : Utilisation **systématique** des **requêtes préparées PDO**, qui séparent les instructions SQL des données utilisateur .
- **Hachage des Mots de Passe** : Utilisation des fonctions natives `password_hash()` et `password_verify()` (Bcrypt) pour sécuriser les mots de passe.
- **Contrôle d'Accès** : Implémentation de "Gardiens de Sécurité" au début de chaque route, vérifiant le rôle de l'utilisateur stocké en `$_SESSION` avant d'autoriser l'accès (`/admin`, `/employe`) .
- **Prévention des failles XSS** : Échappement de toutes les données issues de la base de données avec `htmlspecialchars()` avant leur affichage dans le HTML.
- **Scurisation des Identifiants** : Les mots de passe de la BDD et les clés d'API sont stockés dans un fichier `.env`, lu par la librairie `Dotenv` et ignoré par Git (via `.gitignore`) .

4. Méthodologie de Projet

Gestion des Dépendances

- **Composer (PHP)** a été utilisé pour gérer les librairies backend (PHPMailer, Dotenv).
- Le fichier `composer.lock` est inclus dans Git pour garantir des installations reproductibles lors du déploiement.

 **Gestion de Version (Git)** Pour garantir la stabilité du projet, un workflow Git professionnel a été appliqué:

1. **Branche main** : Reste protégée et ne contient que le code de production stable.
2. **Branche develop** : Sert de branche d'intégration pour les tests et est liée au déploiement sur Railway.
3. **Branches feature/ et hotfix/** : Tout travail (comme l'ajout récent de la documentation) est effectué sur une branche dédiée. Ce travail est ensuite validé via une **Pull Request** avant d'être fusionné dans `develop`, assurant ainsi la qualité du code.

 **Stratégie de Déploiement (PaaS)** L'application est déployée sur **Railway.app**, une plateforme PaaS.

- Le projet est composé de trois services (PHP, MySQL, MongoDB).
- Le service PHP est lié à la branche `develop` du dépôt GitHub.
- Chaque git push sur `develop` déclenche un **déploiement continu** automatique de la nouvelle version.

Partie 3 : Veille et Recherche

Une veille active a été maintenue pour garantir l'utilisation des bonnes pratiques.

1. Veille de Sécurité (OWASP)

* **Source** : Consultation du Top 10 de l'OWASP.

- **Application Concrète** :

Injection : A confirmé la nécessité absolue d'utiliser **PDO et les requêtes préparées**.

Broken Access Control : M'a conduit à implémenter les "Gardiens de Sécurité" (vérification de `$_SESSION[ 'role' ]`) au début de chaque script sensible (admin, employe).

Identification Failures : M'a poussé à utiliser `password_hash()` (BCRYPT) pour les mots de passe.

2. Veille Technologique (Frontend)

* **Source** : Documentation officielle (Chart.js, MDN) et tutoriels.

Application Concrète (Exemple Chart.js) :

Lors de l'implémentation du dashboard admin, j'ai dû afficher plusieurs graphiques dynamiques.

La **documentation de Chart.js** a confirmé qu'il fallait créer plusieurs éléments `<canvas>` distincts et instancier un objet `new Chart()` pour chacun.

Mon script `admin.js` utilise `fetch` pour récupérer les données de l'API `/api/stats`, puis initialise les deux graphiques (Covoiturages par jour et Crédits par jour) avec les données JSON reçues.

Partie 4 : Informations complémentaire

Toute cette documentation est présente sur mon dépôt GIT :

- **Manuel utilisateur** : fourni au format .pdf
- **Documentation technique** : fournie au format .pdf
- **Maquettes** :
 - Wireframes : 3 desktop + 3 mobile
 - Mokups : 3 desktop + 3 mobile
- **Charte graphique** : fournie au format .pdf
- **Base de données** : fichier SQL